Dissecting Packets: From HTTP to TLS

Wireshark Packet Analysis Project Report

Submitted By

- Name: SanthoshS
- Email: thor41361@gmail.com
- Phone: 9150883411

1. Introduction

Wireshark is a powerful network protocol analyzer used to capture and inspect packets traveling across a network. It helps cybersecurity professionals, network administrators, and students understand how communication occurs between systems.

In this project, HTTP, DNS, and TLS traffic were captured and analyzed using Wireshark. The project demonstrates how application-layer protocols operate and how packet-level analysis can help in troubleshooting and security monitoring.

2. Objective

The objective of this project is to analyze real-time network traffic using Wireshark and understand how HTTP requests, DNS resolution, and TLS handshakes appear in captured packets.

3. Tools Used

- Wireshark
- Parrot Security OS
- Web Browser
- Internet Connection

4. Procedure

Step 1 — Launching Wireshark

Wireshark was launched and packet capture was started on the active network interface.

Screenshot to Include:

- Screenshot of Wireshark interface showing active packet capture.

Step 2 — Generating HTTP Traffic

The website `http://neverssl.com` was visited to generate HTTP traffic. Since the website uses HTTP instead of HTTPS, Wireshark was able to capture readable HTTP GET requests.

The following display filter was applied:

http.request.method == "GET"

This filter displayed only HTTP GET request packets.

Observations:

- Source IP Address
- Destination IP Address
- Source Port
- Destination Port (80)
- HTTP Request Method: GET
- Host Information
- Browser User-Agent

5. HTTP Packet Analysis

The HTTP GET packet revealed how the client requested a webpage from the server using port 80. The packet contained request headers including host information, browser details, accepted content types, and connection parameters.

The communication was visible in plain text because HTTP does not encrypt data.

Example Details Observed:

Field	Observation
Source IP	10.0.2.15
Destination IP	34.107.221.82
Destination Port	80
Request Method	GET
Protocol	HTTP

6. DNS Traffic Analysis

A DNS lookup was generated using the command:

nslookup example.com

The following display filter was applied:

dns

Wireshark displayed DNS query and response packets.

Observations:

- Domain name queried
- DNS query packet
- DNS response packet
- Resolved IP address
- Query type (A Record)

7. DNS Packet Analysis

The DNS query packet showed the client requesting the IP address associated with the domain `example.com`. The DNS response packet returned the corresponding IPv4 address.

DNS traffic helps systems convert human-readable domain names into machine-readable IP addresses.

Example Details Observed:

Field	Observation
Queried Domain	example.com
Query Type	AAAA
Resolved IP	2404 : 6800: 4002: 803: : 2003
Protocol	DNS

8. TLS Handshake Analysis

An HTTPS website was visited to generate encrypted traffic:

https://example.com

The following filter was applied:

tls.handshake

Wireshark displayed TLS handshake packets including Client Hello and Server Hello.

Observations:

- TLS version
- Cipher suites
- Server Name Indication (SNI)
- Handshake negotiation details

Screenshot to Include:

- Screenshot showing Client Hello packet.
- Screenshot showing Server Hello packet.
- Screenshot of expanded Transport Layer Security section.
- Screenshot showing TLS version and Cipher Suites.
- Screenshot showing Server Name Indication (SNI).

9. TLS Packet Analysis

The Client Hello packet initiated the secure TLS communication by sending supported TLS versions, cipher suites, and extensions to the server.

The Server Hello packet responded with the selected TLS version and cipher suite to establish encrypted communication.

The Server Name Indication (SNI) extension revealed the hostname requested by the client during the TLS handshake.

Example Details Observed:

Field	Observation
TLS Version	TLS 1.2 / TLS 1.3
Cipher Suite	TLS_AES_128_GCM_SHA256 (0x1301)
SNI	List length = 13 , Type = host_name(0) Name length = 10 Name = medium.com
Protocol	TLS

10. Findings

- HTTP traffic was visible in plain text.
- DNS packets resolved domain names into IP addresses.
- TLS packets encrypted communication between client and server.
- Wireshark filters helped isolate specific protocol traffic.
- Packet analysis provides visibility into network communication and security events.

11. Conclusion

This project provided practical experience in capturing and analyzing network packets using Wireshark. HTTP, DNS, and TLS traffic were successfully inspected using protocol filters.

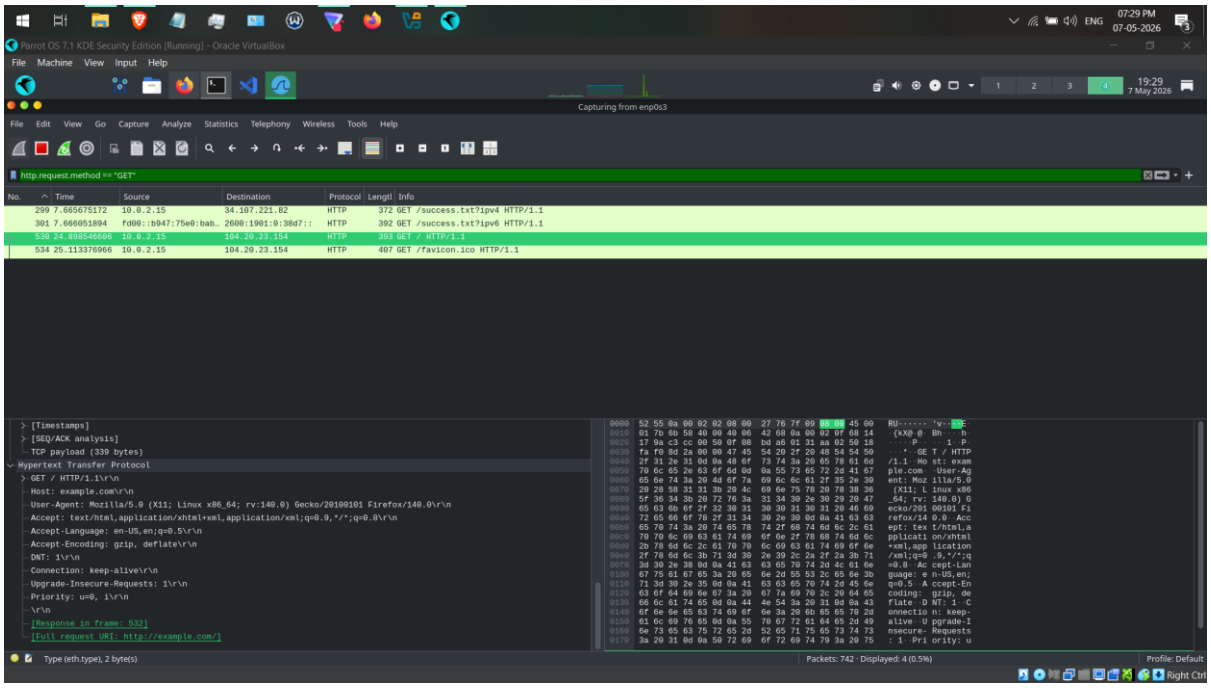
The analysis helped in understanding how web communication occurs across different protocol layers and demonstrated the importance of encrypted communication using TLS.

Wireshark proved to be an effective tool for troubleshooting, protocol inspection, and cybersecurity monitoring.

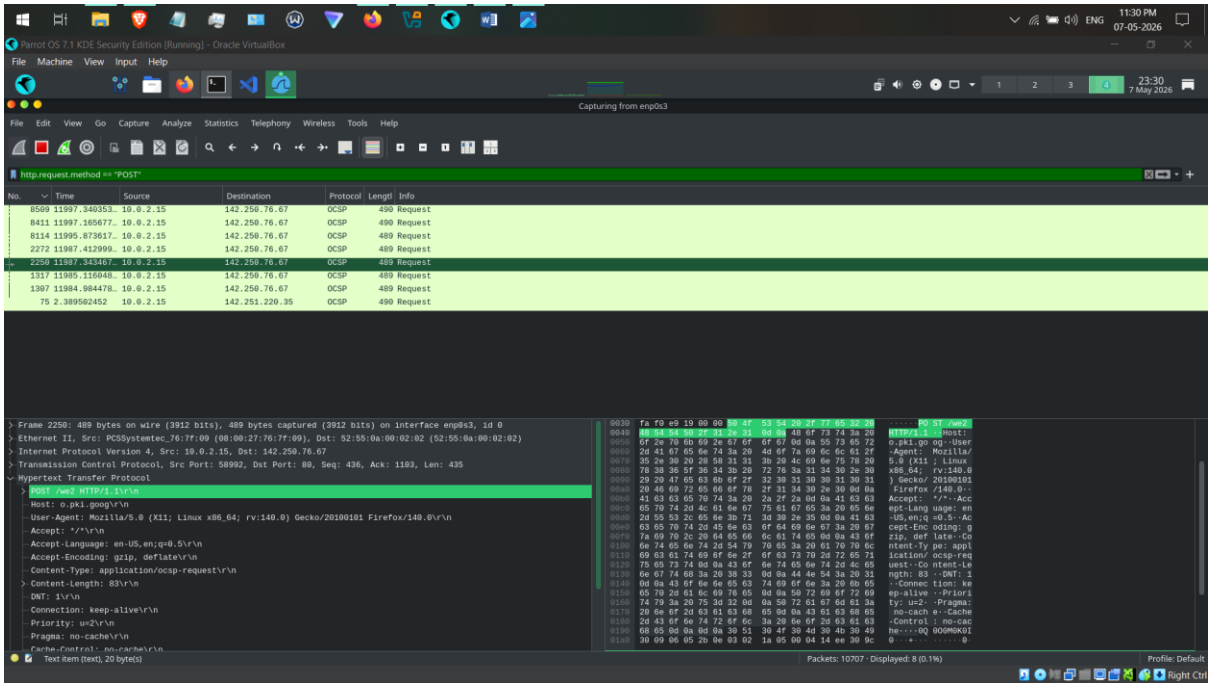
Screenshots

Screenshots of HTTP GET/POST requests.

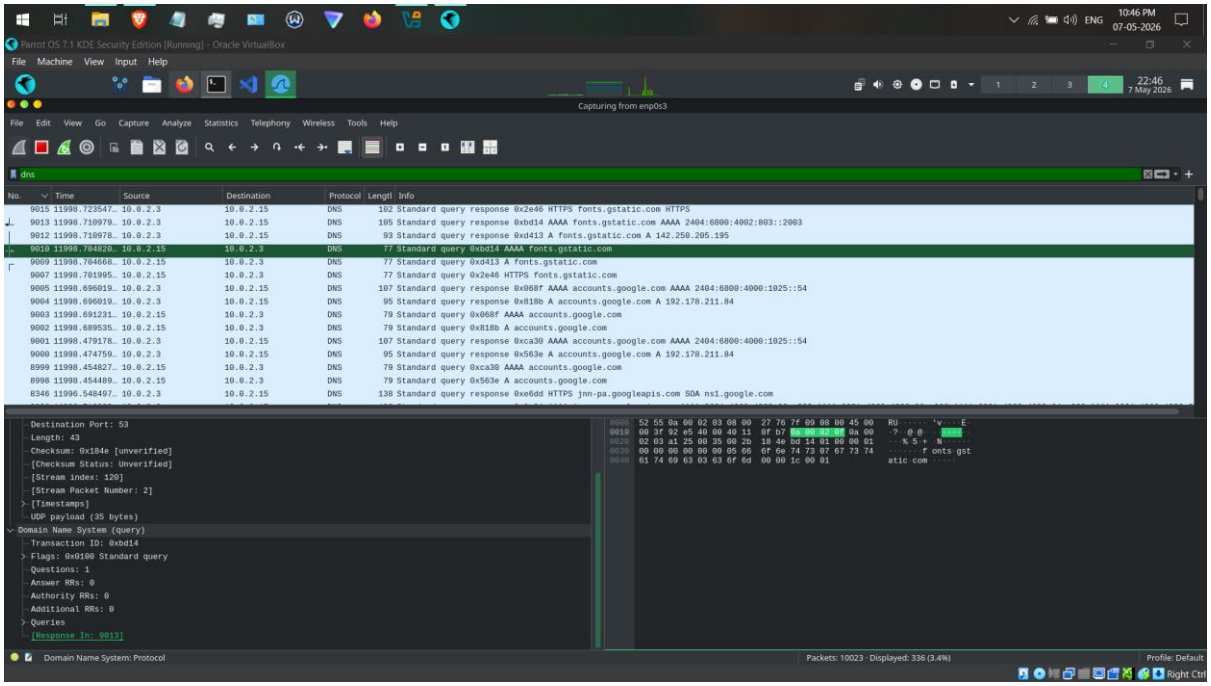
GET


Parrot OS 7.1 KDE Security Edition [Running] - Oracle VirtualBox
File Machine View Input Help
Capturing from enp0s3
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
http.request.method == "GET"
No. Time Source Destination Protocol Length Info
299 7.665675172 10.0.2.15 34.107.221.82 HTTP 372 GET /success.txt?ipv4 HTTP/1.1
301 7.666051894 fd00::b947:75e0:bab… 2600:1901:0:38d7:: HTTP 392 GET /success.txt?ipv6 HTTP/1.1
530 24.898546606 10.0.2.15 104.20.23.154 HTTP 393 GET / HTTP/1.1
534 25.113376966 10.0.2.15 104.20.23.154 HTTP 407 GET /favicon.ico HTTP/1.1
[Timestamps]
[SEQ/ACK analysis]
TCP payload (339 bytes)
Hypertext Transfer Protocol
GET / HTTP/1.1\r\n
Host: example.com\r\n
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0\r\n
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8\r\n
Accept-Language: en-US,en;q=0.5\r\n
Accept-Encoding: gzip, deflate\r\n
DNT: 1\r\n
Connection: keep-alive\r\n
Upgrade-Insecure-Requests: 1\r\n
Priority: u=0, i\r\n
\r\n
[Response in frame: 532]
[Full request URI: http://example.com/]
Type (eth.type), 2 byte(s)
Packets: 742 · Displayed: 4 (0.5%)
Profile: Default

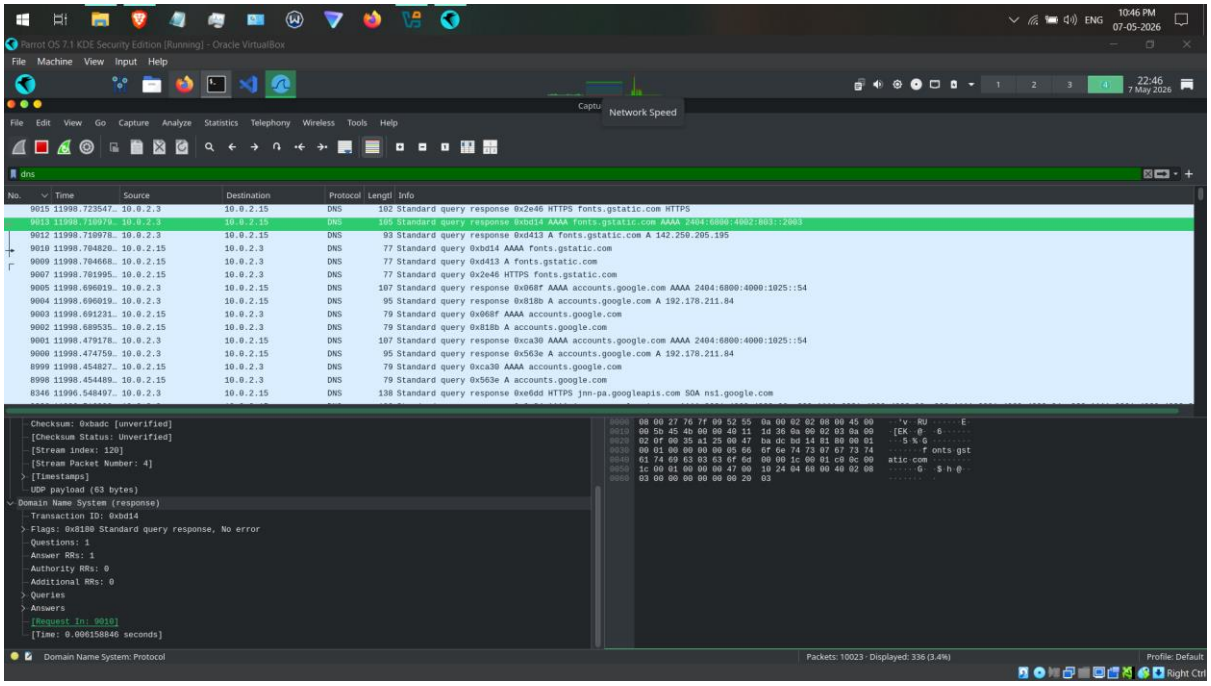
POST


Parrot OS 7.1 KDE Security Edition [Running] - Oracle VirtualBox
File Machine View Input Help
Capturing from enp0s3
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
http.request.method == "POST"
No. Time Source Destination Protocol Length Info
8509 11997.340353… 10.0.2.15 142.250.76.67 OCSP 490 Request
8411 11997.165677… 10.0.2.15 142.250.76.67 OCSP 490 Request
8114 11995.873617… 10.0.2.15 142.250.76.67 OCSP 489 Request
2272 11987.412999… 10.0.2.15 142.250.76.67 OCSP 489 Request
2250 11987.343467… 10.0.2.15 142.250.76.67 OCSP 489 Request
1317 11985.116048… 10.0.2.15 142.250.76.67 OCSP 489 Request
1307 11984.984478… 10.0.2.15 142.250.76.67 OCSP 489 Request
75 2.389502452 10.0.2.15 142.251.220.35 OCSP 490 Request
Frame 2250: 489 bytes on wire (3912 bits), 489 bytes captured (3912 bits) on interface enp0s3, id 0
Ethernet II, Src: PCSSystemtec_76:7f:09 (08:00:27:76:7f:09), Dst: 52:55:0a:00:02:02 (52:55:0a:00:02:02)
Internet Protocol Version 4, Src: 10.0.2.15, Dst: 142.250.76.67
Transmission Control Protocol, Src Port: 58992, Dst Port: 80, Seq: 436, Ack: 1103, Len: 435
Hypertext Transfer Protocol
POST /we2 HTTP/1.1\r\n
Host: o.pki.goog\r\n
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0\r\n
Accept: */*\r\n
Accept-Language: en-US,en;q=0.5\r\n
Accept-Encoding: gzip, deflate\r\n
Content-Type: application/ocsp-request\r\n
Content-Length: 83\r\n
DNT: 1\r\n
Connection: keep-alive\r\n
Priority: u=2\r\n
Pragma: no-cache\r\n
Cache-Control: no-cache\r\n
Text item (text), 20 byte(s)
Packets: 10707 · Displayed: 8 (0.1%)
Profile: Default

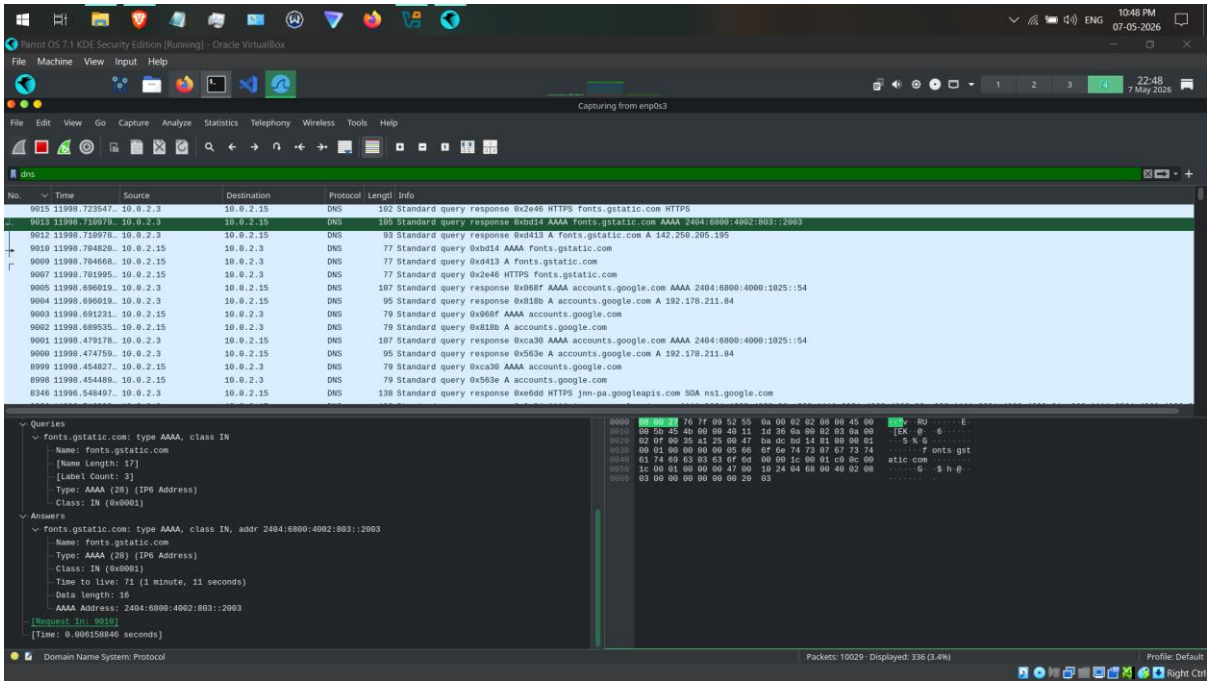
Screenshot of DNS query packet.



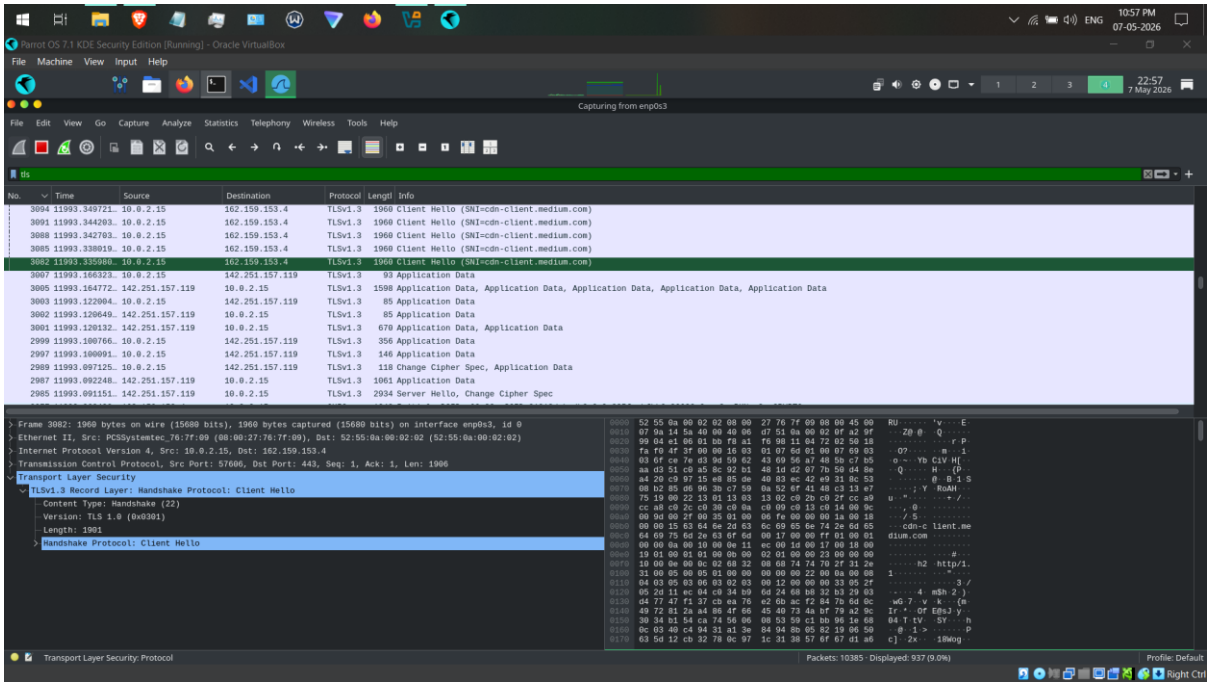
Screenshot of DNS query response packet.



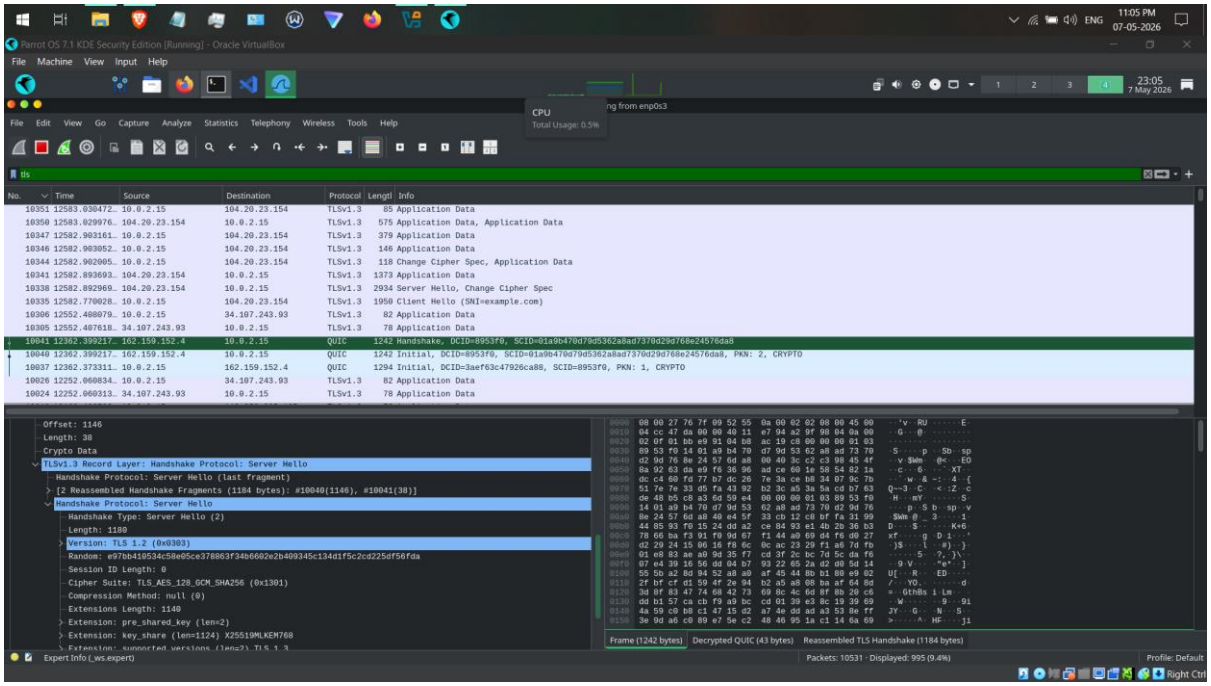
Screenshot of expanded Domain Name System section.



Screenshot showing Client Hello packet.

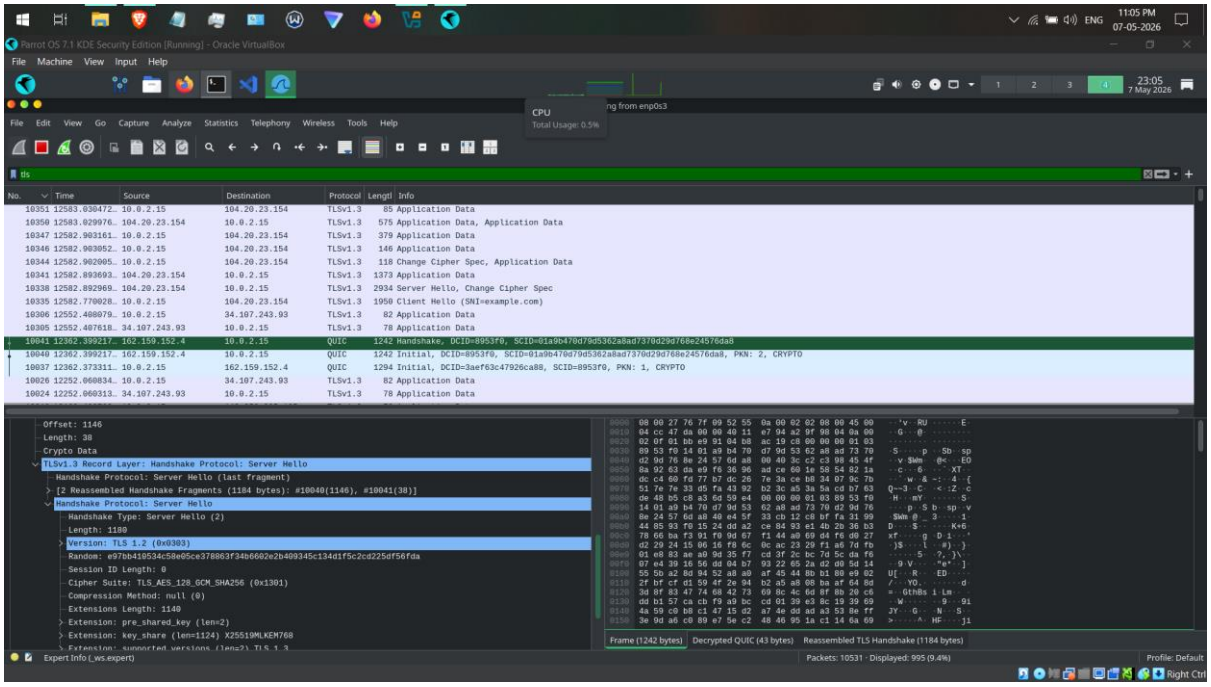


Screenshot showing Server Hello packet.

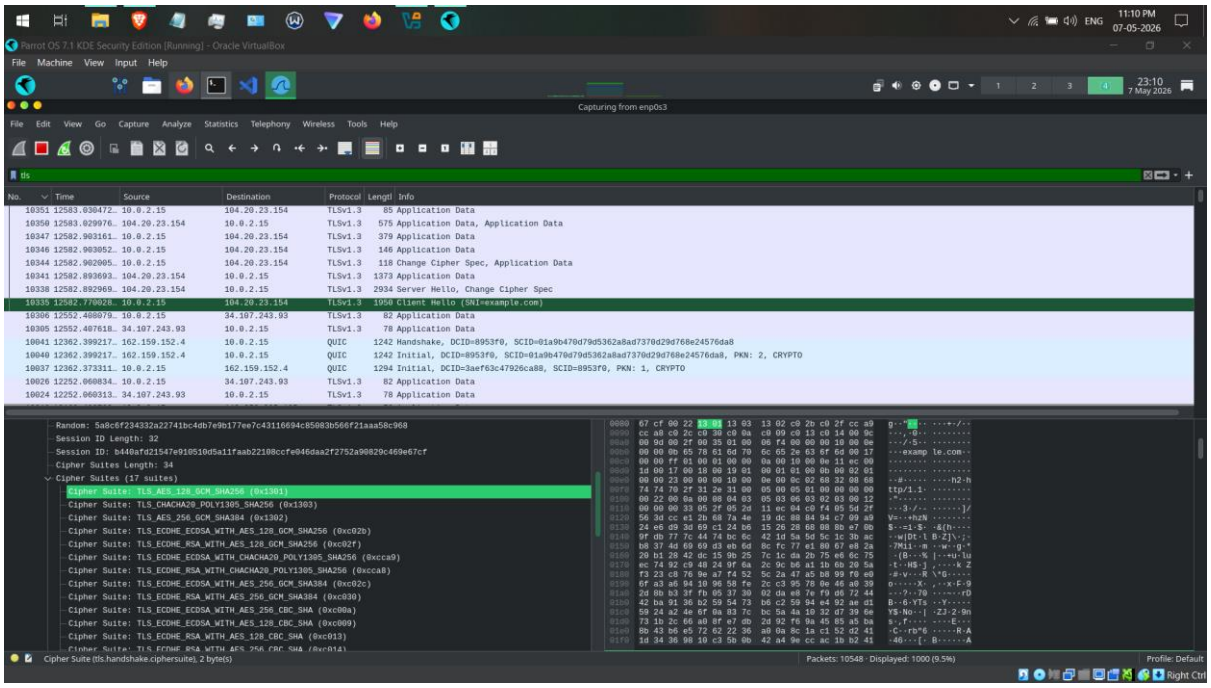
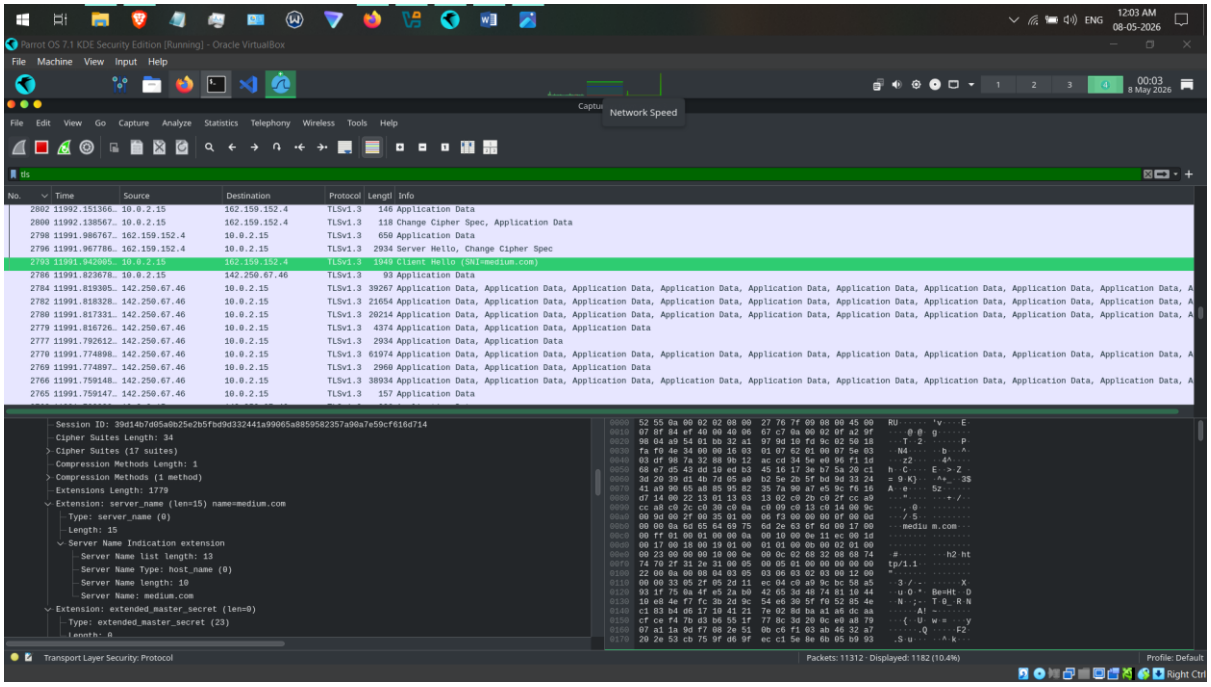


Screenshot showing TLS version and Cipher Suites.

TLS verssion



Cipher Suites



Screenshot showing Server Name Indication (SNI).